

PROGRESS WEEK 11

PRESENTED BY KELOMPOK 4



RINGKASAN EKSEKUTIF

Pada minggu ke-11, tim telah berhasil menyelesaikan pengembangan seluruh mesin utama (core engine) di sisi backend. Seluruh modul kriptografi (SHA-256, AES-256-GCM, dan Enhanced Merkle Tree) telah diimplementasikan dan berhasil melalui tahapan uji fungsionalitas menggunakan Postman. Saat ini, fokus tim sedang bergeser pada penyelesaian User Interface (UI) dan integrasi frontend-backend menggunakan arsitektur API berbasis AJAX.





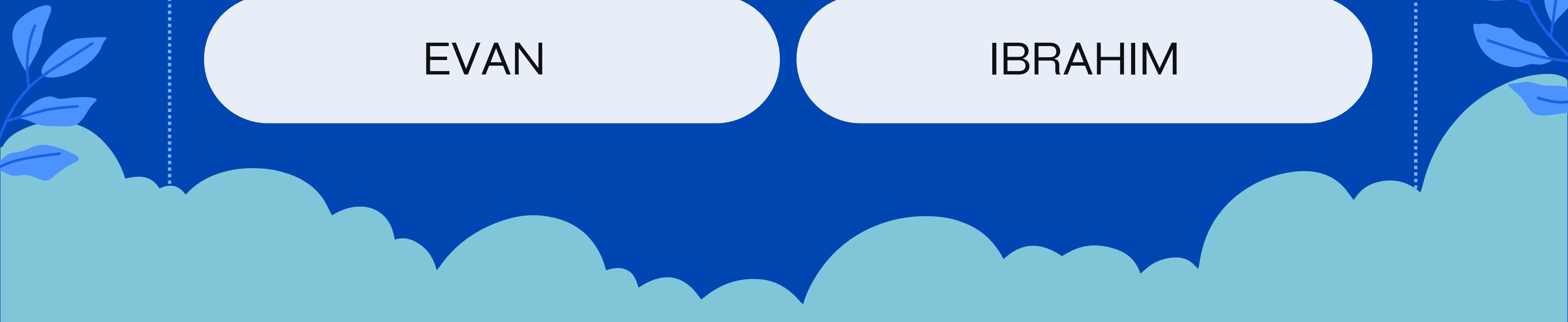
PROGRES INDIVIDU & PENCAPAIAN

TRISHA

JIMLY

EVAN

IBRAHIM



TRISHA

MODUL SHA-256 & DYNAMIC AES KEY

Status: Selesai (100%)

Berhasil mengimplementasikan logika hashing SHA-256 berbasis chunking untuk membaca file secara efisien. Telah merampungkan fungsi generator kunci AES dinamis menggunakan operasi logika XOR antara hash file dan previous hash dari database.



MODUL KRIPTOGRAFI (AES-256-GCM)

```
crypto_aes.py

1  CHUNK_SIZE = 65536 # 64KB
2
3  def hash_file(filepath: str) -> str:
4      if not os.path.exists(filepath):
5          raise FileNotFoundError(f"File {filepath} tidak ditemukan.")
6
7      sha256 = hashlib.sha256()
8      with open(filepath, 'rb') as f:
9          while chunk := f.read(CHUNK_SIZE):
10             sha256.update(chunk)
11     return sha256.hexdigest()
```

Menghitung hash unik dari dokumen.

Membaca file secara bertahap (per 64KB) agar hemat memori/RAM, menghasilkan identitas 256-bit mutlak untuk file tersebut.

MODUL KRIPTOGRAFI (AES-256-GCM)

```
crypto_aes.py

1 def xor_hex(hex1: str, hex2: str) -> bytes:
2     # XOR dua string hex dan return bytes (32 byte = 256 bit)
3     b1 = bytes.fromhex(hex1)
4     b2 = bytes.fromhex(hex2)
5     return bytes(a ^ b for a, b in zip(b1, b2))
```

Fungsi pembantu (helper) operasi kriptografi tingkat bit.
Menyilangkan (XOR) dua nilai hexadecimal dan mengubah hasilnya menjadi format bytes murni.

MODUL KRIPTOGRAFI (AES-256-GCM)

```
crypto_aes.py

1  def generate_dynamic_key(file_hash: str, prev_hash: str) -> bytes:
2      # Generate AES-256 key dari XOR hash file dan prev hash
3      key = xor_hex(file_hash, prev_hash)
4
5      if len(key) < 32:
6          key = key.ljust(32, b'\0')
7      else:
8          key = key[:32]
9
10     return key
```

Membuat kunci rahasia AES (256-bit) sekali pakai.
Memanfaatkan fungsi xor_hex untuk menyilangkan hash dokumen saat ini dengan hash dokumen sebelumnya.

MODUL KRIPTOGRAFI (AES-256-GCM)

```
crypto_aes.py

1 def encrypt_file(input_path: str, key: bytes, output_path: str) -> str:
2     if not os.path.exists(input_path):
3         raise FileNotFoundError(f"File {input_path} tidak ditemukan.")
4
5     nonce = os.urandom(12)
6     encryptor = Cipher(algorithms.AES(key), modes.GCM(nonce)).encryptor()
7
8     with open(input_path, 'rb') as f_in, open(output_path, 'wb') as f_out:
9         f_out.write(nonce)
10
11         while chunk := f_in.read(CHUNK_SIZE):
12             f_out.write(encryptor.update(chunk))
13
14         encryptor.finalize()
15         f_out.write(encryptor.tag)
16
17     return output_path
```

Mengunci dokumen menjadi ciphertext tak terbaca. Menggunakan algoritma AES-256-GCM. Menyematkan angka acak (Nonce) di awal dan segel keaslian (Auth Tag) di akhir file.

MODUL KRIPTOGRAFI (AES-256-GCM)

```
crypto_aes.py

1 def decrypt_file(input_path: str, key: bytes, output_path: str) -> str:
2     if not os.path.exists(input_path):
3         raise FileNotFoundError(f"File {input_path} tidak ditemukan.")
4
5     file_size = os.path.getsize(input_path)
6     if file_size < 28:
7         raise ValueError("File corrupted: Terlalu kecil untuk AES-GCM.")
8
9     with open(input_path, 'rb') as f_in:
10         nonce = f_in.read(12)
11
12         f_in.seek(-16, os.SEEK_END)
13         tag = f_in.read(16)
14
15         f_in.seek(12)
16         decryptor = Cipher(algorithms.AES(key), modes.GCM(nonce, tag)).decryptor()
17
18         with open(output_path, 'wb') as f_out:
19             bytes_to_read = file_size - 28
20             while bytes_to_read > 0:
21                 chunk = f_in.read(min(CHUNK_SIZE, bytes_to_read))
22                 if not chunk: break
23                 f_out.write(decryptor.update(chunk))
24                 bytes_to_read -= len(chunk)
25
26         decryptor.finalize()
27
28     return output_path
```

Membuka dokumen kembali ke bentuk aslinya.

Membaca Nonce dan kunci. Jika Auth Tag tidak cocok (indikasi data telah diotak-atik), sistem otomatis membatalkan proses (tamper-proof).

RINGKASAN

- file_hash (Output: 64 Karakter Teks Hex): Menghitung sidik jari file pakai SHA-256. Diubah 1 spasi saja, 64 karakternya berubah total.
- dynamic_key (Output: 32 Bytes Biner murni): Hasil file_hash XOR prev_hash.
- encrypt_file (Output: nama_file.bin): Membungkus data dengan urutan: Buat Nonce (12 byte) ditaruh di paling depan -> Ubah teks asli jadi Ciphertext (acak) -> Tambah Auth Tag (16 byte) di paling akhir sebagai segel.
- decrypt_file (Proses Validasi):
 - 1.Normal: Dokumen diunduh, sistem membaca Nonce dan kunci. Karena Auth Tag utuh, file .bin berhasil dikembalikan ke bentuk aslinya.
 - 2.Hacker: Coba menyusup dan menyisipkan/mengubah 1 huruf di file .bin. Sistem langsung mendeteksi segel Auth Tag rusak, menggagalkan dekripsi, dan menampilkan pesan error (Tamper-Proof).

JIMLY MODUL ENHANCED MERKLE TREE

Status: Selesai (100%)

Logika pembentukan pohon Merkle (bottom-up) telah selesai. Sistem kini mampu memecah ciphertext menjadi blok berukuran 1KB, melakukan hashing pada tiap blok, dan menghitung Merkle Root yang akan dijadikan segel integritas di cloud.



IMPLEMENTASI DARI MERKLE TREE PADA PROGRAM

```
import hashlib
import os

CHUNK_SIZE = 1024 # memotong ciphertext per 1kb

def sha256_hash(data):
    # untuk hashing data bytes or string
    if isinstance(data, str):
        data = data.encode()
    return hashlib.sha256(data).hexdigest()
```

- Fungsi utilitas dasar pembentukan pohon hash untuk fase 4 dan 5

IMPLEMENTASI DARI MERKLE TREE PADA PROGRAM

```
# membentuk merkle tree (fase 4)
def build_merkle_tree(ciphertext_path):
    if not os.path.exists(ciphertext_path):
        raise FileNotFoundError(f"file {ciphertext_path} tidak ditemukan")

    level = []

    # chunking
    with open(ciphertext_path, 'rb') as f:
        while True:
            chunk = f.read(CHUNK_SIZE)
            if not chunk:
                break
            level.append(sha256_hash(chunk)) # Langsung di-hash dan dimasukkan ke level

    if not level:
        return sha256_hash(b'')

    # susun bottom up
    while len(level) > 1:
        next_level = []
        for i in range(0, len(level), 2):
            hash1 = level[i]
            hash2 = level[i+1] if i + 1 < len(level) else hash1

            # gabung hash
            combined_hash = sha256_hash(hash1 + hash2)
            next_level.append(combined_hash)

        level = next_level

    return level[0]
```

Fungsi fase 4 (Pemantapan Integritas)

- File dipotong menjadi balok-balok kecil berukuran 1 KB
- Setiap potongan langsung di-hash menggunakan fungsi SHA-256 sebelumnya. Hasilnya menjadi baris terbawah dalam struktur pohon
- Kemudian, baris tersebut di-hash kembali secara berpasangan dan membentuk pohon merkle. Proses berhenti ketika hanya tersisa satu hash di puncak
- Hasil akhirnya adalah nilai hash di puncak pohon merkle (64 karakter hexadecimal) yang berfungsi sebagai bukti validitas file

IMPLEMENTASI DARI MERKLE TREE PADA PROGRAM

```
# audit cloud & verif (fase 5)
def verify_data_integrity(ciphertext_path, stored_merkle_root):
    try:
        recalculated_root = build_merkle_tree(ciphertext_path)
    except FileNotFoundError:
        return {
            "is_valid": False,
            "status": "CORRUPTED",
            "message": "Gagal: Ciphertext tidak ditemukan di public cloud"
        }

    # pencocokan data
    if recalculated_root == stored_merkle_root:
        return {
            "is_valid": True,
            "status": "Valid",
            "message": "Data identik"
        }
    else:
        return {
            "is_valid": False,
            "status": "CORRUPTED",
            "message": "Data tidak cocok. Ciphertext telah dimodifikasi"
        }
```

Fungsi fase 5 (Audit & Verifikasi)

- Sistem mengambil ciphertext dari penyimpanan dan menghitung ulang Merkle Root-nya untuk menyesuaikan dengan yang tercatat di Blockchain
- Jika file tidak ditemukan, sistem langsung memberikan status CORRUPTED
- Jika hasil hitung ulang identik dengan merkle root asli, sistem mengembalikan status Valid
- Jika tidak, mengembalikan status CORRUPTED

EVAN MODUL ENKRIPSI & EVOLUSI BLOCKCHAIN

Status: Selesai (100%)

Modul eksekusi enkripsi/dekripsi menggunakan algoritma AES-256-GCM telah berjalan stabil. Melakukan transisi strategis dari prototipe `blockchain_sim.py` (simulasi in-memory) menjadi sistem Anchoring Ledger permanen yang terintegrasi langsung dengan relasional database di Supabase.



BLOCK STRUCTURE

```
# BLOCK STRUCTURE
class Block:
    def __init__(self, index, file_hash, prev_hash, ciphertext_path):
        self.index = index
        self.timestamp = time.time()
        self.file_hash = file_hash
        self.prev_hash = prev_hash
        self.ciphertext_path = ciphertext_path
        self.block_hash = self.calculate_hash()

    def calculate_hash(self):
        data = f"{self.index}{self.timestamp}{self.file_hash}{self.prev_hash}{self.ciphertext_path}"
        return hashlib.sha256(data.encode()).hexdigest()

    def to_dict(self):
        return {
            "index": self.index,
            "timestamp": self.timestamp,
            "file_hash": self.file_hash,
            "prev_hash": self.prev_hash,
            "ciphertext_path": self.ciphertext_path,
            "block_hash": self.block_hash
        }
```

Bagian ini digunakan untuk membuat struktur data sebuah block pada blockchain.

Setiap block menyimpan:

- index → nomor urut block
- timestamp → waktu pembuatan block
- file_hash → hash file asli
- prev_hash → hash block sebelumnya
- ciphertext_path → lokasi file terenkripsi
- block_hash → hash unik block

Fungsi calculate_hash() digunakan untuk menghasilkan hash SHA-256 berdasarkan seluruh isi block sehingga integritas data dapat dijaga.

Jika isi block berubah, maka hash juga akan berubah sehingga manipulasi data dapat terdeteksi.

Fungsi to_dict() digunakan untuk mengubah data block menjadi format dictionary agar mudah ditampilkan atau disimpan ke database.

BLOCKCHAIN STRUCTURE

```
# BLOCKCHAIN
class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis_block = Block(
            index=0,
            file_hash="0" * 64,
            prev_hash="0" * 64,
            ciphertext_path="genesis"
        )
        self.chain.append(genesis_block)

    def get_last_block(self):
        return self.chain[-1]

    def add_block(self, file_hash, ciphertext_path):
        prev_block = self.get_last_block()

        new_block = Block(
            index=len(self.chain),
            file_hash=file_hash,
            prev_hash=prev_block.block_hash,
            ciphertext_path=ciphertext_path
        )

        self.chain.append(new_block)
        return new_block
```

Bagian ini digunakan untuk mengelola seluruh rantai blockchain.

Sistem dimulai dengan membuat genesis block sebagai block pertama.

Fungsi utama:

- `add_block()` → menambahkan block baru ke blockchain
- `get_last_block()` → mengambil block terakhir
- `is_chain_valid()` → memvalidasi integritas blockchain
- `print_chain()` → menampilkan seluruh isi blockchain

Setiap block baru akan menyimpan hash dari block sebelumnya (`prev_hash`) sehingga seluruh block saling terhubung membentuk rantai data yang aman.

Fungsi validasi digunakan untuk memastikan:

- data block tidak berubah
- hubungan antar block tetap valid
- manipulasi data dapat terdeteksi secara otomatis

IBRAHIM INTERFACE & INTEGRASI MODUL

Status: On-Progress (50%)

Kerangka antarmuka interaktif sedang dibangun dan saat ini sedang dalam tahap akhir menghubungkan UI dengan endpoint API (menggunakan Fetch API) agar upload file, animasi pipeline, dan audit dapat dilakukan langsung melalui browser.



DASHBOARD CONSOLE

● CloudGuard EMT: ACTIVE

Awaiting Input...

Input Data

Pilih file dari komputer lalu kirim ke backend untuk diproses oleh modul `core\_engine`.

⬆

Drag file ke sini atau klik untuk memilih file.
Mendukung semua jenis file. File akan diproses dengan modul SHA-256, AES, Merkle, dan disimpan.

Pilih File

Process Encryption

Pipeline Status

Status eksekusi modul `core\_engine` ditampilkan secara berurutan.

Upload
FILE SELECTION

SHA-256
HASHING

AES Key
DYNAMIC KEY

Merkle
ROOT

Store
DB VAULT

SHA-256 HASH

-

DYNAMIC AES KEY

-

MERKLE ROOT

-

Audit & Verification

Fase 5: Verifikasi integritas ciphertext yang tersimpan di cloud.

Verify Integrity

Simulate Tampering

INTEGRASI API (1)

```
try {  
  const response = await fetch('/api/security/upload/', {  
    method: 'POST',  
    body: formData  
  });
```

Bagian ini berfungsi untuk mengirimkan file ke server

INTEGRASI API (2)

```
async function verifyIntegrity() {
  if (!window.currentDocId) return;
  showAuditResult('Memeriksa integritas file... Harap tunggu.', 'info');
  try {
    const response = await fetch(`/api/security/audit/${window.currentDocId}/`, { method: 'GET' });
    const result = await response.json();
    if (!response.ok || result.status !== 'Sukses') {
      showAuditResult(`Verifikasi gagal: ${result.message || 'Server error.'}`, 'error');
      return;
    }
    const valid = result.audit_detail?.is_valid;
    if (valid) {
      showAuditResult('Integritas valid. Ciphertext terverifikasi aman.', 'success');
    } else {
      showAuditResult('Integritas rusak! Sistem mendeteksi perubahan pada ciphertext.', 'error');
    }
  } catch (err) {
    showAuditResult('Gagal memeriksa integritas. Cek koneksi atau server.', 'error');
  }
}
```

Bagian ini berfungsi untuk memverifikasi integritas secara real-time

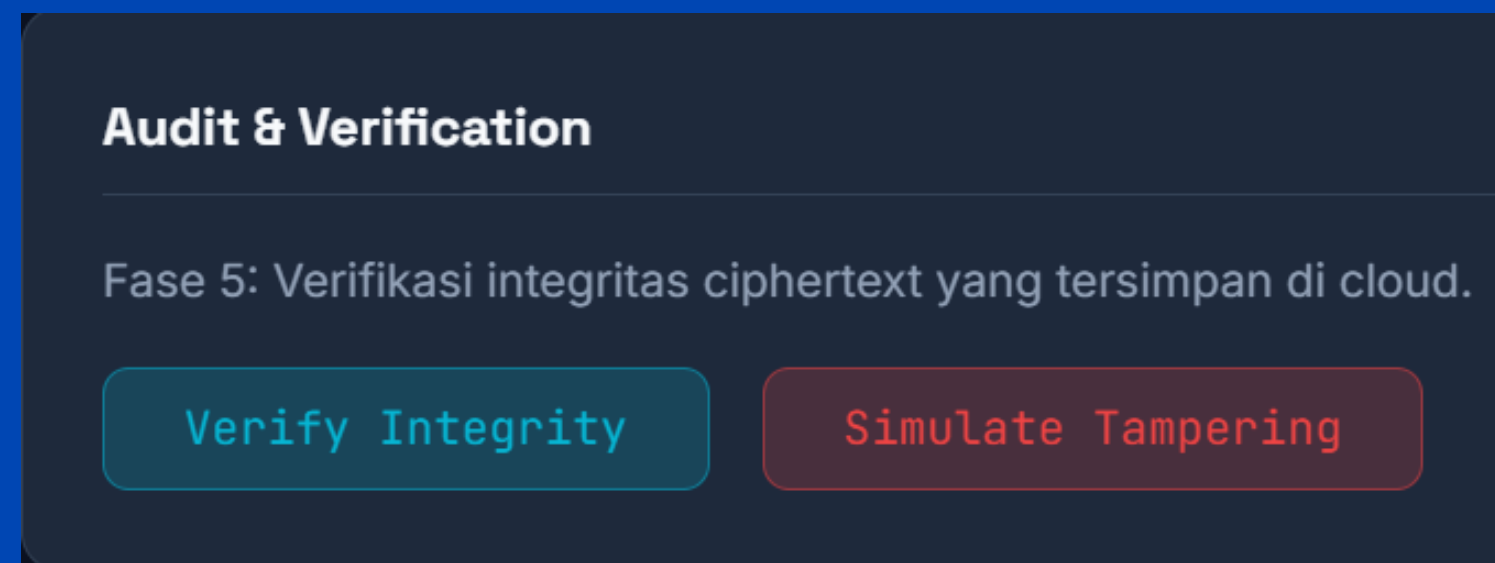
MODUL VERIFIKASI (1)

```
<div class="bg-surface border border-border p-6 rounded-xl flex flex-col gap-4">
  <h3 class="font-display font-bold text-heading border-b border-border pb-2">Audit & Verification</h3>
  <p class="text-body text-sm">Fase 5: Verifikasi integritas ciphertext yang tersimpan di cloud.</p>

  <div class="flex gap-4">
    <button id="btn-verify" class="px-6 py-2 bg-secondary/20 text-secondary border border-secondary/50 rounded-lg hover:bg-secondary/30 transition-all font-mono text-sm opacity-50 cursor-not-allowed" disabled>
      Verify Integrity
    </button>
    <button id="btn-tamper" class="px-6 py-2 bg-danger/20 text-danger border border-danger/50 rounded-lg hover:bg-danger/30 transition-all font-mono text-sm opacity-50 cursor-not-allowed" disabled>
      Simulate Tampering
    </button>
  </div>

  <div id="audit-result" class="hidden p-4 rounded-lg border font-mono text-sm mt-2 transition-all"></div>
</div>
```

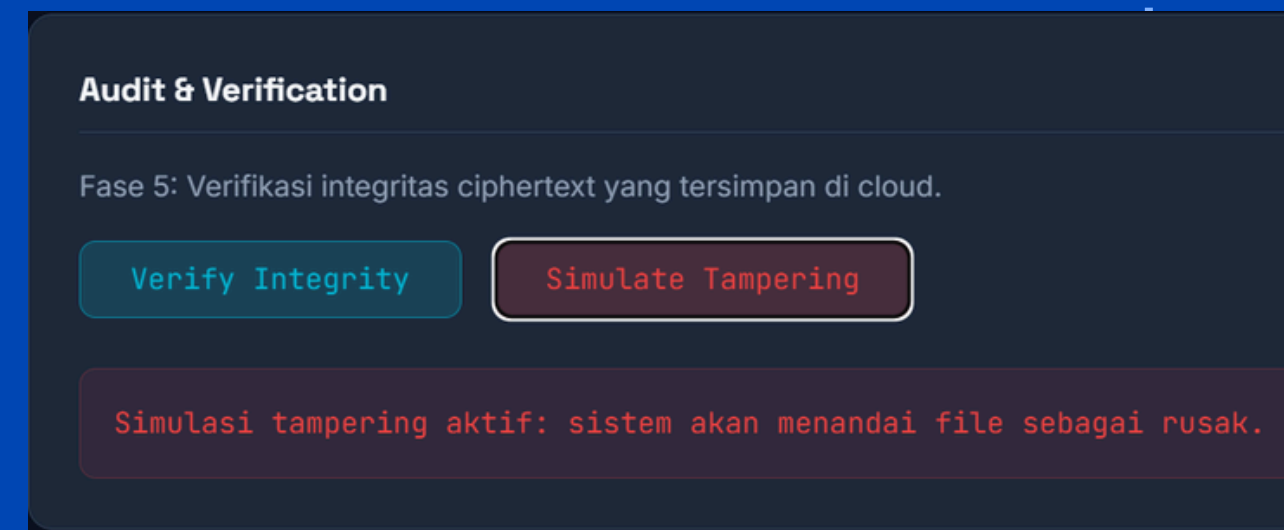
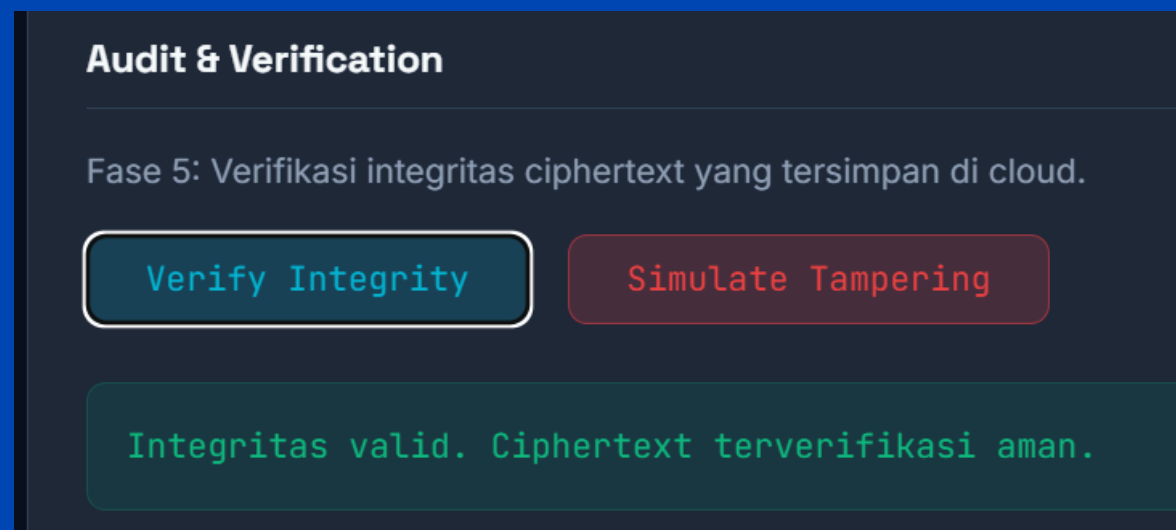
Bagian ini berfungsi untuk menampilkan tombol “Verify Integrity” dan “Simulate Tempering”



MODUL VERIFIKASI (2)

```
function showAuditResult(message, variant = 'info') {  
  const colors = {  
    info: 'text-body/80 bg-background border-border',  
    success: 'text-primary bg-emerald-500/10 border-emerald-500/20',  
    error: 'text-danger bg-red-500/10 border-red-500/20',  
  };  
  auditResult.textContent = message;  
  auditResult.className = `p-4 rounded-lg border font-mono text-sm mt-2 transition-all ${colors[variant]} || colors.info`;  
  auditResult.classList.remove('hidden');  
}
```

Bagian ini berfungsi untuk memberikan warna hijau jika valid dan merah jika data terdeteksi dimanipulasi.



UPLOAD BOX

Ini merupakan bagian untuk tempat use menaruh file

```
if request.method == 'POST' and request.FILES.get('document'):  
    uploaded_file = request.FILES['document']  
    file_name = uploaded_file.name  
    file_size = uploaded_file.size
```

```
<div id="drop-zone" class="border-2 border-dashed border-border rounded-3xl p-10 text-center cursor-pointer hover:border-primary transition-colors bg-background/50">  
  <div class="flex flex-col items-center justify-center gap-3">  
    <div class="w-12 h-12 rounded-2xl bg-surface border border-border flex items-center justify-center">  
      <svg class="w-6 h-6 text-primary" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 16v1a3 3 0 00 3h10a3 3 0 00 3v-1m-4-8l-4-4m0 0L8 8m4-4v12"/></svg>  
    </div>  
    <div class="text-left max-w-[260px]">  
      <p id="drop-text" class="text-body text-sm font-mono">Drag file ke sini atau klik untuk memilih file.</p>  
      <p class="text-xs text-muted">Mendukung semua jenis file. File akan diproses dengan modul SHA-256, AES, Merkle, dan disimpan.</p>  
    </div>  
    <label for="file-input" class="inline-flex items-center gap-2 px-4 py-2 rounded-full border border-border text-sm text-primary cursor-pointer hover:bg-primary/10 transition-colors">  
      Pilih File  
    </label>  
  </div>  
  <input type="file" id="file-input" class="hidden">  
</div>
```

Input Data

Pilih file dari komputer lalu kirim ke backend untuk diproses oleh modul `core\_engine`.



Drag file ke sini atau klik untuk memilih file.

Mendukung semua jenis file. File akan diproses dengan modul SHA-256, AES, Merkle, dan disimpan.

Pilih File

Process Encryption

MODUL YANG DIINTEGRASI (1)

Hashing

Begitu file diterima, sistem langsung menjalankan SHA-256 untuk membuat fingerprint unik dari file asli.

```
<div id="stage-sha" class="console-stage-card">
  <div class="stage-title">SHA-256</div>
  <div class="stage-subtitle">Hashing</div>
</div>
```

```
file_hash = hash_file(temp_path)

if SecureDocument.objects.filter(file_hash=file_hash).exists():
    if os.path.exists(temp_path):
        os.remove(temp_path)
    return JsonResponse({
        "status": "Gagal",
        "message": "File ini sudah pernah diamankan di dalam sistem CloudGuard EMT!"
    }, status=400)

last_doc = SecureDocument.objects.order_by('-block_index').first()

if last_doc:
    prev_hash = last_doc.file_hash
    next_index = last_doc.block_index + 1
else:
    prev_hash = "0" * 64
    next_index = 1
```


MODUL YANG DIINTEGRASI (2)

Dynamic Key

Sistem membuat kunci AES secara otomatis yang unik untuk setiap file.

Kunci ini dibuat dengan menggabungkan hash file saat ini dengan hash dari file sebelumnya (konsep Blockchain Chaining).

```
key = generate_dynamic_key(file_hash, prev_hash)
```

```
<div id="stage-key" class="console-stage-card">  
  <div class="stage-title">AES Key</div>  
  <div class="stage-subtitle">Dynamic Key</div>  
</div>
```


MODUL YANG DIINTEGRASI (3)

Enkripsi

Mengubah file asli menjadi Ciphertext (.bin) menggunakan algoritma AES-256-CBC sehingga isi file tidak bisa dibaca tanpa kunci yang tepat.

```
def encrypt_file(input_path: str, key: bytes, output_path: str) -> str:
    if not os.path.exists(input_path):
        raise FileNotFoundError(f"File {input_path} tidak ditemukan.")

    nonce = os.urandom(12)
    encryptor = Cipher(algorithms.AES(key), modes.GCM(nonce)).encryptor()

    with open(input_path, 'rb') as f_in, open(output_path, 'wb') as f_out:
        f_out.write(nonce)

        while chunk := f_in.read(CHUNK_SIZE):
            f_out.write(encryptor.update(chunk))

        encryptor.finalize()
        f_out.write(encryptor.tag)

    return output_path
```

MODUL YANG DIINTEGRASI (4)

Merkle Tree

Setelah file dienkrupsi, sistem menghitung Merkle Root. Ini digunakan sebagai segel keamanan untuk memastikan bahwa file di cloud storage tidak pernah dimanipulasi.

```
merkle_root = build_merkle_tree(encrypted_temp_path)
```

```
<div id="stage-merkle" class="console-stage-card">  
  <div class="stage-title">Merkle</div>  
  <div class="stage-subtitle">Root</div>  
</div>
```

MODUL YANG DIINTEGRASI (5)

Cloud Storage

Menghubungkan aplikasi dengan server Supabase untuk menyimpan file terenkripsi secara permanen di cloud.

```
file_name_supabase = f"{file_name}.bin"
supabase.storage.from_(settings.SUPABASE_BUCKET).upload(
    path=file_name_supabase,
    file=encrypted_temp_path,
    file_options={"content-type": "application/octet-stream"}
)
```

```
<div id="stage-store" class="console-stage-card">
  <div class="stage-title">Store</div>
  <div class="stage-subtitle">DB Vault</div>
</div>
```

HASIL HASHING, DYNAMIC KEY, MERKLE ROOT

SHA-256 HASH

beb19a6bde7ee81af96bd08229a6eee4a2178f1dc25eb5fcf956e7bad6bb38df

DYNAMIC AES KEY

[DIKUNCI & TIDAK DISIMPAN]

MERKLE ROOT

fbb0dfcb93e559eac4012742c6cb7ab824201fe520d28e8e88404bd6359d0c5f

```
<div class="grid grid-cols-1 gap-3 font-mono text-xs">
  <div class="bg-background border border-border p-4 rounded-xl">
    <div class="text-muted text-[10px] uppercase tracking-widest mb-1">SHA-256 Hash</div>
    <div id="out-hash" class="text-primary break-all">-</div>
  </div>
  <div class="bg-background border border-border p-4 rounded-xl">
    <div class="text-muted text-[10px] uppercase tracking-widest mb-1">Dynamic AES Key</div>
    <div id="out-key" class="text-secondary break-all">-</div>
  </div>
  <div class="bg-background border border-border p-4 rounded-xl">
    <div class="text-muted text-[10px] uppercase tracking-widest mb-1">Merkle Root</div>
    <div id="out-merkle" class="text-primary break-all">-</div>
  </div>
</div>
```

ANIMASI STATUS PIPELINE

```
function setConsoleStage(activeStage) {
  [stageUpload, stageSha, stageKey, stageMerkle, stageStore].forEach((stage) => {
    stage.classList.remove('active');
  });

  if (activeStage === 'upload') stageUpload.classList.add('active');
  if (activeStage === 'sha') stageSha.classList.add('active');
  if (activeStage === 'key') stageKey.classList.add('active');
  if (activeStage === 'merkle') stageMerkle.classList.add('active');
  if (activeStage === 'store') stageStore.classList.add('active');
}

function completeConsoleStage(stage) {
  const mapping = {
    upload: stageUpload,
    sha: stageSha,
    key: stageKey,
    merkle: stageMerkle,
    store: stageStore,
  };
  const el = mapping[stage];
  if (el) {
    el.classList.add('complete');
    el.classList.remove('active');
  }
}
```

Pipeline Status

Status eksekusi modul `core\_engine` ditampilkan secara berurutan.

Upload

FILE SELECTION

SHA-256

HASHING

AES Key

DYNAMIC KEY

Merkle

ROOT

Store

DB VAULT

Pipeline Status

Status eksekusi modul `core\_engine` ditampilkan secara berurutan.

Upload

FILE SELECTION

SHA-256

HASHING

AES Key

DYNAMIC KEY

Merkle

ROOT

Store

DB VAULT

REFERENSI

Ma, X., Yang, W., Zhu, Y., & Bai, Z. (2022). A secure and efficient data deduplication scheme with dynamic ownership management in cloud computing. arXiv preprint arXiv:2208.09030. <https://doi.org/10.48550/arXiv.2208.09030>

Shakor, M. Y., Khaleel, M. I., Safran, M., Alfarhood, S., & Zhu, M. (2024). Dynamic AES encryption and blockchain key management: A novel solution for cloud data security. IEEE Access, 12. <https://doi.org/10.1109/ACCESS.2024.3351119>

Tian, J., Wang, H., & Wang, M. (2021). Data integrity auditing for secure cloud storage using user behavior prediction. Computers & Security, 105, 102245. <https://doi.org/10.1016/j.cose.2021.102245>

Wang, Y. (2024). Data structure and privacy protection analysis in big data environment based on blockchain technology. International Journal of Intelligent Networks, 5, 120–132. <https://doi.org/10.1016/j.ijin.2024.02.005>

TERIMA KASIH

